Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI
Email: vtu29748@veltech.edu.in
Roll no: vtu29748
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 7_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 38.5

Section 1 : Coding

1. Problem Statement

Bob is working on a project involving network optimization. He needs a program to find the Minimum Spanning Tree (MST) of a given undirected weighted graph with 5 nodes (numbered 0 to 4).

The graph is represented as a 5×5 symmetric adjacency matrix, where a value of 0 indicates no direct connection between two nodes. Implement Prim's Algorithm to find the MST and print each edge with its weight.

Example

Input

0 3 0 5 0

4 0 2 3 5

0 3 0 8 7

3 8 0 0 9

0 5 7 2 0

Output

Edge Weight

0 - 1 4

1 - 2 3

1 - 3 8

1 - 4 5

Explanation

In the given 5x5 matrix representing an undirected graph, Bob has specified the weights of edges. The Prim's algorithm is applied to find the Minimum Spanning Tree (MST). The output displays the edges along with their corresponding weights, forming the optimal tree connecting all vertices. In this case, the MST includes edges (0-1), (1-2), (1-3), and (1-4) with weights 4, 3, 8, and 5 respectively, ensuring minimal total weight for network optimization.

Input Format

The input consists of 5 lines, each containing 5 space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two nodes. The matrix is symmetric (graph[i][j] = graph[j][i]) since the graph is undirected.

Output Format

The first line of output prints the header: 'Edge \tWeight'

Each of the next 4 lines prints one MST edge in the format:

'u - v \tw'

where u is the parent node, v is the child node, and w is the integer edge weight, printed in ascending order of child node index (node-index order).

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0 2 0 6 0
2 0 3 8 5
0 3 0 8 7
6 8 0 0 9
0 5 7 9 0

Output: Edge Weight
0 - 1 2
1 - 2 3
0 - 3 6
1 - 4 5

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

#define V 5

int minKey(int key[], bool mstSet[]) {
    int min = INT_MAX, min_index;
    for (int v = 0; v < V; v++)
        if (mstSet[v] == false && key[v] < min)
            min = key[v], min_index = v;
    return min_index;
}

void primMST(int graph[V][V]) {
    int parent[V];
    int key[V];
    bool mstSet[V];
```

```
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = false;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet);
        mstSet[u] = true;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printf("Edge \tWeight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
    }
}

int main() {
    int graph[V][V];
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (scanf("%d", &graph[i][j]) != 1) return 0;
        }
    }

    primMST(graph);

    return 0;
}
```

Status : Correct *Marks : 10/10*

2. Problem Statement

You are given a set of cities in a country and the cost of building railway track segments to connect them. Your task is to find the minimum cost required to build the railway tracks and connect all the cities. The cost of each track segment is given, along with the source city and destination city to which it connects.

Write a program that takes the number of cities, the number of track segments, and the details of each track segment as input. Implement Kruskal's Algorithm to calculate the minimum cost of building the railway tracks.

Cities are numbered 0 to N-1.

Example

Input:

3

3

0 1 4

1 2 5

0 2 3

Output:

7

Explanation:

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4.Track Segment 2: Connects city 1 and city 2 with a weight of 5.Track Segment 3: Connects city 0 and city 2 with a weight of 3.The algorithm finds the

minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is 3 + 4 = 7.

Input Format

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

Output Format

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
5
0 1 1
0 2 4
1 2 2
1 3 5
2 3 3
Output: 6

Answer

```
#include <stdio.h>
```

```c
#include <stdlib.h>

typedef struct {
    int u, v, w;
} Edge;

int parent[100];

int find(int i) {
    if (parent[i] != i)
        parent[i] = find(parent[i]);
    return parent[i];
}

void unionSet(int u, int v) {
    int pu = find(u);
    int pv = find(v);
    parent[pu] = pv;
}

int cmp(const void *a, const void *b) {
    return ((Edge *)a)->w - ((Edge *)b)->w;
}

int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);

    Edge edges[100];

    for (int i = 0; i < m; i++)
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);

    for (int i = 0; i < n; i++)
        parent[i] = i;

    qsort(edges, m, sizeof(Edge), cmp);

    int count = 0, cost = 0;

    for (int i = 0; i < m; i++) {
```

```
            int u = edges[i].u;
            int v = edges[i].v;

            if (find(u) != find(v)) {
                unionSet(u, v);
                cost += edges[i].w;
                count++;
            }
        }

        if (count == n - 1)
            printf("%d", cost);
        else
            printf("It is not possible to connect all cities.");

        return 0;
    }
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Raja is given a map of cities connected by bidirectional roads, where each road has an associated construction cost (weight). His task is to find the Minimum Spanning Tree (MST) of this network using Prim's Algorithm, starting from city 0, so that all cities are connected with the minimum total construction cost.

Prim's Algorithm works as follows:

Start from the source city (city 0). Mark it as included in the MST.At each step, select the edge with the minimum weight that connects a city already in the MST to a city not yet in the MST.Repeat until all cities are included in the MST.

The program should display each MST edge (the city included and the cost to connect it) and the total MST cost.

Note: The given graph is a connected, undirected, weighted graph. Self-loops and parallel edges are not present.

Input Format

The first line of input consists of two space-separated integers, n and m, where n represents the number of cities (vertices) and m represents the number of roads (edges).

The next m lines each consist of three space-separated integers u, v, and w, where u and v are the two cities connected by the road (0-indexed) and w is the construction cost (weight) of the road.

Output Format

The output consists of n lines.

The first n-1 lines each print the MST edge added, in the format:

"Edge: u - v, Cost: w", where u is the city already in the MST, v is the newly added city, and w is the integer edge weight. Edges are printed in the order they are added by Prim's algorithm starting from city 0.

The last line prints "Total MST Cost: t", where t is the total integer cost of the minimum spanning tree.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 5
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4
Output: Edge: 0 - 1, Cost: 10
Edge: 3 - 2, Cost: 4

Edge: 0 - 3, Cost: 5
Total MST Cost: 19

Answer

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

#define MAX_V 100

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return 0;

    long long graph[MAX_V][MAX_V];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graph[i][j] = 0;
        }
    }

    for (int i = 0; i < m; i++) {
        int u, v;
        long long w;
        scanf("%d %d %lld", &u, &v, &w);
        graph[u][v] = w;
        graph[v][u] = w;
    }

    long long key[MAX_V];
    int parent[MAX_V];
    bool mstSet[MAX_V];

    for (int i = 0; i < n; i++) {
        key[i] = LLONG_MAX;
        mstSet[i] = false;
    }

    key[0] = 0;
    parent[0] = -1;
    long long totalCost = 0;
```

```
    for (int count = 0; count < n; count++) {
        long long min = LLONG_MAX;
        int u = -1;

        for (int v = 0; v < n; v++) {
            if (!mstSet[v] && key[v] < min) {
                min = key[v];
                u = v;
            }
        }

        if (u == -1) break;

        mstSet[u] = true;
        totalCost += key[u];

        if (parent[u] != -1) {
            printf("Edge: %d - %d, Cost: %lld\n", parent[u], u, key[u]);
        }

        for (int v = 0; v < n; v++) {
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printf("Total MST Cost: %lld\n", totalCost);

    return 0;
}
```

Status : Partially correct **Marks : 1/10**

4. Problem Statement

Riya is working as a data scientist in a company that deals with large text data. Due to storage and bandwidth limitations, she needs a way to compress text while maintaining its readability.

She decides to use Huffman Encoding, a well-known algorithm that assigns shorter binary codes to frequently occurring characters and longer binary codes to less common characters, thus reducing the overall message size.

Given a set of characters with their respective frequencies, help Riya generate Huffman Codes for each character. Then, using these codes, encode a given message.

Assign '0' to left branches and '1' to right branches, merge the two nodes with the lowest frequency at each step. Clarify that the message will only contain characters from the given input set. Specify that Huffman codes are printed in the order they appear during in-order/pre-order traversal of the Huffman tree (matching sample output)

The message will only contain characters from the provided character set.

Input Format

The first line of input consists of an integer n, representing the number of unique characters.

The next n lines each consist of a character (an uppercase or lowercase English letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the message to be encoded. The message will only contain characters from the provided character set.

Output Format

The first n lines of output print the Huffman code for each character in the format:

"character: code"

where the characters are printed in the order they appear as leaf nodes during the Huffman tree traversal (left child before right child, assigning '0' to left and '1' to right).

All n input characters and their codes are printed, regardless of whether they appear in the message.

The last line of output prints the encoded message as a sequence of 0s and 1s obtained by replacing each character in the message with its corresponding Huffman code.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3
A 1
B 2
C 3
ABCCBAABBC
Output: C: 0
A: 10
B: 11
10110011101011110

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
};

struct Node* createNode(char ch, int freq) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

struct Node* nodes[100];
int size = 0;
```

```
void insert(struct Node* node) {
    nodes[size++] = node;
}

void sort() {
    for (int i = 0; i < size - 1; i++)
        for (int j = i + 1; j < size; j++)
            if (nodes[i]->freq > nodes[j]->freq) {
                struct Node* temp = nodes[i];
                nodes[i] = nodes[j];
                nodes[j] = temp;
            }
}

struct Node* extractMin() {
    sort();
    return nodes[--size];
}

void buildHuffman() {
    while (size > 1) {
        sort();
        struct Node* left = nodes[0];
        struct Node* right = nodes[1];

        for (int i = 2; i < size; i++)
            nodes[i - 2] = nodes[i];
        size -= 2;

        struct Node* merged = createNode('\0', left->freq + right->freq);
        merged->left = left;
        merged->right = right;

        insert(merged);
    }
}

char codes[256][256];

void generateCodes(struct Node* root, char* code, int depth) {
    if (!root) return;
```

```
    if (!root->left && !root->right) {
        code[depth] = '\0';
        strcpy(codes[(int)root->ch], code);
        printf("%c: %s\n", root->ch, code);
        return;
    }

    code[depth] = '0';
    generateCodes(root->left, code, depth + 1);

    code[depth] = '1';
    generateCodes(root->right, code, depth + 1);
}

int main() {
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        char ch;
        int freq;
        scanf(" %c %d", &ch, &freq);
        insert(createNode(ch, freq));
    }

    buildHuffman();

    char code[256];
    generateCodes(nodes[0], code, 0);

    char message[300];
    scanf("%s", message);

    for (int i = 0; message[i]; i++)
        printf("%s", codes[(int)message[i]]);

    return 0;
}
```

Status : Partially correct **Marks : 7.5/10**

5. Problem Statement

In a clandestine world of spies and covert operations, a group of operatives uses Huffman coding to encode and decode their classified messages securely.

Huffman coding assigns shorter binary codes to more frequently occurring characters and longer codes to less frequent ones. The algorithm works as follows:

Each character and its frequency is inserted into a min-heap (priority queue).At each step, extract the two nodes with the lowest frequencies, merge them into a new internal node whose frequency is the sum of the two, and reinsert it into the heap.Repeat until only one node (the root) remains — this is the Huffman tree.Assign '0' to every left branch and '1' to every right branch. The binary string from root to each leaf node is that character's Huffman code.

Given a set of characters with their frequencies, build the Huffman tree, print the codes for all characters, encode the given message using these codes, and decode the encoded message back to the original.

Note: The message will only contain characters from the provided input set. All character frequencies are unique.

Input Format

The first line of input consists of a positive integer N, representing the number of unique characters.

The next N lines each consist of a character (an uppercase or lowercase English letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the secret message to be encoded. The message contains only characters from the provided character set

Output Format

The first N lines of output each print the Huffman code for a character in the format:

"character: code"

where characters are printed in the order they appear as leaf nodes during Huffman tree traversal (left child before right child, with '0' assigned to left and '1' to right).

The next line prints the encoded message — a sequence of 0s and 1s obtained by replacing each character in the message with its Huffman code.

The last line prints the decoded message obtained by traversing the Huffman tree using the encoded bits, which must match the original input message.

Refer to the sample outputs for better understanding and formatting.

Sample Test Case

Input: 5
A 5
B 9
C 12
D 13
E 16
ABCDE
Output: C: 00
D: 01
A: 100
B: 101
E: 11
100101000111
ABCDE

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char ch;
    int freq;
```

```
    struct Node *left, *right;
};

struct Node* createNode(char ch, int freq) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

struct Node* heap[100];
int size = 0;

void swap(int i, int j) {
    struct Node* temp = heap[i];
    heap[i] = heap[j];
    heap[j] = temp;
}

void heapifyUp(int i) {
    while (i && heap[(i - 1) / 2]->freq > heap[i]->freq) {
        swap(i, (i - 1) / 2);
        i = (i - 1) / 2;
    }
}

void heapifyDown(int i) {
    int smallest = i;
    int l = 2 * i + 1;
    int r = 2 * i + 2;

    if (l < size && heap[l]->freq < heap[smallest]->freq)
        smallest = l;
    if (r < size && heap[r]->freq < heap[smallest]->freq)
        smallest = r;

    if (smallest != i) {
        swap(i, smallest);
        heapifyDown(smallest);
    }
}
```

```
void insert(struct Node* node) {
    heap[size] = node;
    heapifyUp(size);
    size++;
}

struct Node* extractMin() {
    struct Node* root = heap[0];
    heap[0] = heap[--size];
    heapifyDown(0);
    return root;
}

struct Node* buildHuffman() {
    while (size > 1) {
        struct Node* left = extractMin();
        struct Node* right = extractMin();

        struct Node* merged = createNode('\0', left->freq + right->freq);
        merged->left = left;
        merged->right = right;

        insert(merged);
    }
    return heap[0];
}

char codes[256][256];

void generateCodes(struct Node* root, char* code, int depth) {
    if (!root) return;

    if (!root->left && !root->right) {
        code[depth] = '\0';
        strcpy(codes[(int)root->ch], code);
        printf("%c: %s\n", root->ch, code);
        return;
    }

    code[depth] = '0';
    generateCodes(root->left, code, depth + 1);
```

```
    code[depth] = '1';
    generateCodes(root->right, code, depth + 1);
}

int main() {
    int n;
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        char ch;
        int freq;
        scanf(" %c %d", &ch, &freq);
        insert(createNode(ch, freq));
    }

    struct Node* root = buildHuffman();

    char code[256];
    generateCodes(root, code, 0);

    char message[100];
    scanf("%s", message);

    char encoded[1000] = "";
    for (int i = 0; message[i]; i++)
        strcat(encoded, codes[(int)message[i]]);

    printf("%s\n", encoded);

    struct Node* curr = root;
    for (int i = 0; encoded[i]; i++) {
        if (encoded[i] == '0')
            curr = curr->left;
        else
            curr = curr->right;

        if (!curr->left && !curr->right) {
            printf("%c", curr->ch);
            curr = root;
        }
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***